

IV. Module Intelligence Artificielle — Conception et Architecture

IV.1 Positionnement du module IA dans le système

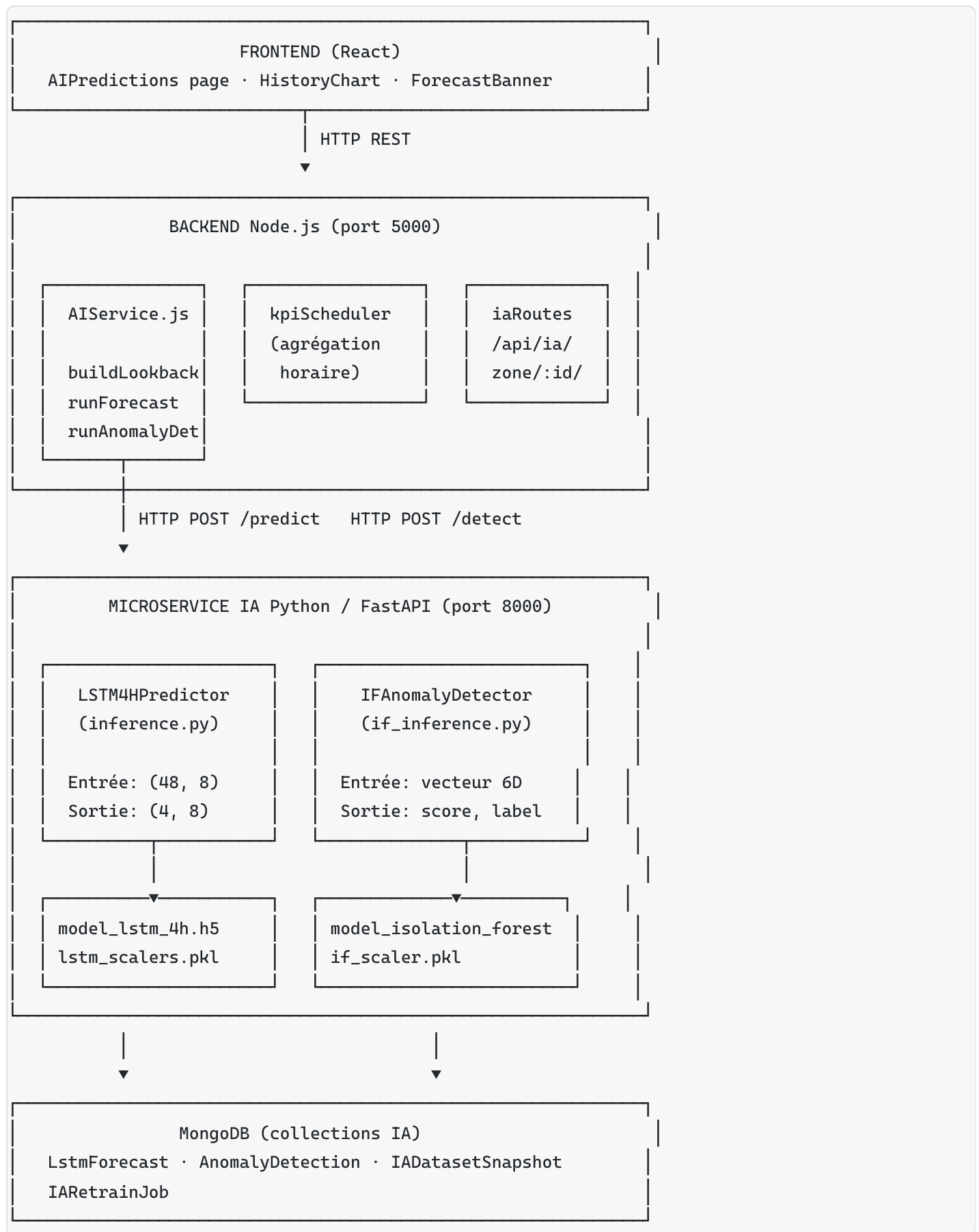
IV.1.1 Rôle et complémentarité avec le moteur d’alertes

Le système comporte deux mécanismes d’analyse distincts et complémentaires. Le moteur d’alertes du backend Node.js réagit en temps réel à un dépassement de seuil réglementaire constaté : il est déterministe, instantané, et ne nécessite pas d’historique. Le module IA, lui, adopte une posture **anticipatoire et statistique** :

Dimension	Moteur d’alertes (backend)	Module IA
Nature	Réactive — seuil dépassé maintenant	Proactive — dépassement prévu dans 4h
Détection	Seuil fixe (VLE réglementaire)	Profil anormal sans seuil franchi
Données requises	Une seule mesure	48 heures d’historique
Technologie	Node.js, règles métier	Python, TensorFlow, scikit-learn
Latence	< 100 ms	~100 ms (inférence)
Cas d’usage	Alerte opérationnelle immédiate	Planification de shift, maintenance préventive

Le module IA est donc un **deuxième niveau de vigilance** : il peut déclencher une alerte de type `Forecast` ou `Anomaly` avant que le moteur de seuil ne soit atteint.

IV.1.2 Architecture globale du module



IV.2 Choix technologique — Étude comparative

IV.2.1 Choix du langage Python

Le module IA est un microservice Python indépendant, découplé du backend Node.js. Ce choix repose sur la maturité de l'écosystème Python pour le machine learning : TensorFlow/Keras, scikit-learn, pandas et numpy constituent le standard de facto pour les réseaux de neurones récurrents et les algorithmes de détection d'anomalies. Aucun équivalent Node.js n'atteint le même niveau de maturité pour les réseaux LSTM.

IV.2.2 Choix du framework FastAPI

L'API d'inférence est exposée via **FastAPI** (Python, port 8000). Le tableau suivant compare les alternatives :

Critère	FastAPI ✓	Flask	Django REST
Performance I/O	ASGI asynchrone — haute performance	WSGI synchrone — bloquant	WSGI — overhead
Validation automatique	Pydantic — schémas typés	Manuelle	DRF serializers
Documentation auto	OpenAPI/Swagger intégré	Extension tierce	DRF navigable API
Latence inférence	Faible (async)	Correcte	Lourde
Démarrage projet	Immédiat	Immédiat	Lourd
Verdict	Retenu	Alternative viable	Surdimensionné

FastAPI est retenu pour sa validation de données automatique via Pydantic, sa documentation OpenAPI intégrée, et ses performances I/O non bloquantes — pertinent quand l'inférence TensorFlow occupe le thread Python pendant ~90 ms.

IV.2.3 Choix des modèles ML

Le système utilise deux algorithmes complémentaires qui couvrent des cas d'usage distincts :

Critère	LSTM (Long Short-Term Memory)	Isolation Forest
Famille	Réseau de neurones récurrent	Ensemble d'arbres de décision
Objectif	Prédiction de séries temporelles	Détection d'anomalies multivariées
Données requises	Séquences ordonnées dans le temps	Vecteur d'état instantané
Cas d'usage	Anticiper un dépassement futur	Détecter un profil actuel anormal
Interprétabilité	Faible (boîte noire)	Modérée (score d'isolation)
Complexité entraînement	Élevée (GPU conseillé)	Faible (CPU, quelques secondes)
Latence inférence	~90 ms	~20 ms

IV.3 Pipeline de prétraitement des données

Avant tout entraînement ou inférence, les lectures brutes de MongoDB doivent être transformées en un tenseur numérique exploitable. Le pipeline de prétraitement est documenté dans `preprocessor.py` et appliqué systématiquement.

IV.3.1 Conception du pipeline (6 étapes)

Lectures brutes MongoDB (collection readings)



Étape 1 : RÉÉCHANTILLONNAGE (resampling)

Conversion des timestamps irréguliers (capteurs ~30 s)
vers une grille horaire uniforme (1 mesure/heure)
agrégation par moyenne sur la fenêtre



Étape 2 : INTERPOLATION DES LACUNES

Si NaN consécutifs $\leq 5\%$ de la série → interpolation linéaire
Si NaN consécutifs $> 5\%$ → marquer comme indisponible



Étape 3 : SUPPRESSION DES OUTLIERS PHYSIQUES

Z-score : $|Z| > 3,5$ → valeur suspecte, remplacée par interpolation
IQR : valeur hors $[Q1 - 3 \times IQR, Q3 + 3 \times IQR]$ → suspecte
Bornes physiques absolues définies par polluant dans config.py
(ex : CO₂ : [300, 5000] ppm – valeur impossible hors plage)



Étape 4 : LISSAGE EXPONENTIEL

Moyenne mobile exponentielle (EMA, $\alpha = 0,3$)
Atténue le bruit capteur haute fréquence sans décaler la phase



Étape 5 : INGÉNIERIE DE FEATURES

Fenêtre glissante 10 min (rolling_mean_10m)
Fenêtre glissante 30 min (rolling_mean_30m)
Corrélation croisée CO₂ ↔ NO_x (fenêtre 5 min)
→ ces features capturent les dynamiques locales et les couplages



Étape 6 : NORMALISATION

MinMaxScaler [0, 1] pour le LSTM (fit uniquement sur le train set)
StandardScaler ($\mu=0$, $\sigma=1$) pour l'Isolation Forest
Les scalers sont sérialisés (joblib) avec les modèles pour assurer
la cohérence entre entraînement et inférence



Tenseur prêt pour entraînement ou inférence

Règle critique : séparation temporelle stricte. Le scaler est ajusté (`.fit()`) uniquement sur le jeu d'entraînement, puis appliqué (`.transform()`) sur validation et test sans refitting. Cette règle empêche la fuite de

données temporelles (data leakage) qui biaiserait les métriques de performance.

IV.3.2 Winsorisation anti-outliers (LSTM)

Avant normalisation, une **winsorisation** est appliquée sur les colonnes polluants : les valeurs inférieures au 1er percentile ou supérieures au 99e percentile du jeu d'entraînement sont écrêtées. Cette technique, plus robuste que la suppression, préserve les tendances extrêmes légitimes (pics industriels réels) tout en neutralisant les aberrations de capteur isolées.

IV.4 Modèle LSTM — Conception de la prédiction temporelle

IV.4.1 Justification du choix LSTM

Les concentrations de polluants industriels présentent deux caractéristiques qui motivent l'utilisation d'un réseau LSTM plutôt qu'un modèle statistique classique (ARIMA, régression linéaire) :

1. **Dépendances à long terme** : les émissions d'une cimenterie suivent des cycles shift (3×8h), des cycles hebdomadaires (réduction weekend), et des variations saisonnières. Un LSTM peut mémoriser ces patterns grâce à ses cellules de mémoire à long terme.
2. **Corrélations multivariées** : CO₂ et NO_x sont couplés par la combustion du four — une hausse simultanée est plus significative qu'une hausse isolée. Le LSTM traite tous les polluants comme un vecteur d'état à chaque pas de temps, capturant ces interdépendances.

IV.4.2 Architecture du réseau

```
Entrée : (48, 8)
| 48 pas horaires (= 48 heures de lookback = 2 cycles jour/nuit)
| 8 polluants : CO2, NOx, SOx, PM2.5, PM10, COV, Température, Humidité
|
▼
LSTM Layer 1 – 64 cellules, return_sequences=True, activation=ReLU
| Apprentissage des patterns court terme (heures)
| return_sequences=True : transmet la séquence complète à L2
|
▼
Dropout(0.2)
| Régularisation : désactive 20% des neurones aléatoirement
| Préviend le surapprentissage sur le jeu d'entraînement
|
▼
LSTM Layer 2 – 32 cellules, activation=ReLU
| Apprentissage des patterns plus longs (cycles jour/nuit)
| Entrée : séquence complète depuis L1
|
▼
BatchNormalization
| Stabilise les gradients, accélère la convergence
|
▼
Dropout(0.2)
|
▼
Dense Layer – 16 neurones, activation=ReLU
| Combinaison des représentations apprises
|
▼
Dense Layer – 32 neurones (4 horizons × 8 polluants)
|
▼
Reshape(4, 8)
|
▼
Sortie : (4, 8)
| 4 pas horaires futurs (+1h, +2h, +3h, +4h)
| 8 valeurs de polluants normalisées [0, 1]
```

IV.4.3 Stratégie d'entraînement

Cadence temporelle. Les données brutes (mesures toutes les 30 s) sont agrégées en **agrégats horaires** avant l'entraînement. Cette granularité horaire est alignée sur celle des KPI schedulers du backend (collections `AggregateData`), ce qui permet de réutiliser les données déjà calculées pour l'inférence en production.

Partitionnement temporel. La division train/validation/test respecte l'ordre chronologique strict : 70% train → 15% validation → 15% test. Il n'y a aucun mélange aléatoire — mélanger des données futures avec des données passées introduirait une fuite temporelle qui gonflerait artificiellement les métriques.

Fonction de perte pondérée. La fonction de perte Huber pondérée par polluant oriente l'apprentissage vers les polluants réglementairement prioritaires :

Polluant	Poids	Raison
CO ₂	2,0	Indicateur principal de combustion
PM ₁₀	2,0	Réglementation stricte (santé)
Température	1,2	Contexte météo important
COV, Humidité	0,5	Variabilité modérée
NO _x , SO _x , PM _{2.5}	0,3	Corrélés à CO ₂ (redondance partielle)

La perte Huber ($\delta = 0,05$) est préférée au MSE car elle est robuste aux outliers : elle se comporte comme le MSE pour les petites erreurs et comme le MAE pour les grandes erreurs, évitant que quelques pics industriels ne dominant la mise à jour des gradients.

Early stopping sur skill score. L'arrêt anticipé n'est pas déclenché par la décroissance de la val_loss, mais par le **val_skill** — la métrique de skill score définie comme :

$$skill = 1 - \frac{MAE_{LSTM}}{MAE_{persistence}}$$

$$skill = 1 - \frac{MAE_{persistence}}{MAE_{LSTM}}$$

Un skill positif signifie que le LSTM fait mieux que la prédiction naïve (répéter la dernière valeur). L'entraînement s'arrête quand le skill ne s'améliore plus après 15 epochs, ce qui est plus significatif métier qu'une simple décroissance de perte.

IV.4.4 Mécanisme hybride LSTM + Persistance

Une décision de conception importante est le **fallback intelligent** par polluant : si le skill score d'un polluant est négatif ou nul en entraînement (LSTM moins précis que la persistance), la prédiction finale pour ce polluant utilise la persistance (dernière valeur observée répétée) au lieu de la sortie LSTM.

Pour chaque polluant P et chaque horizon +h :

```

skill(P) > 0 → prediction = LSTM(P)
skill(P) ≤ 0 → prediction = last_value(P)
P ∈ FALLBACK_LIST → prediction = last_value(P)
0 < α < 1 → prediction = α × LSTM + (1-α) × pers

```

L'humidité relative, dont la dynamique dépend de la météo extérieure non capturée par le réseau de capteurs, est systématiquement en fallback persistance.

IV.4.5 Critères go/no-go de déploiement

Un rapport de skill complet est généré après chaque entraînement. Le déploiement est conditionné à des seuils minimum :

Critère	Seuil minimum	Résultat obtenu	Statut
Skill global	$\geq 0,02$	0,0665	✓ PASS
Skill CO ₂	$\geq 0,05$	0,0587	✓ PASS
Skill PM ₁₀	$\geq 0,08$	0,2414	✓ PASS
Ratio MAE +1h (LSTM/pers)	$\leq 1,15$	0,945	✓ PASS

Décision : `go_deploy = true` avec fallback persistance sur l'humidité.

IV.5 Modèle Isolation Forest — Conception de la détection d'anomalies

IV.5.1 Principe et justification

L'Isolation Forest est un algorithme d'apprentissage non supervisé de la famille des méthodes d'ensemble. Son principe repose sur la propriété suivante : **isoler une anomalie nécessite moins de coupes aléatoires qu'isoler un point normal**, car les anomalies sont éloignées des points denses.

Ce choix est justifié par trois contraintes du projet :

1. **Absence d'étiquettes d'anomalies en production** : les données industrielles tunisiennes ne disposent pas d'un historique d'anomalies labellisées permettant un apprentissage supervisé.
2. **Détection multivariée** : un pic de NO_x isolé sans augmentation correspondante de CO₂ est une anomalie (incohérence du profil de combustion), mais un seuil monovarié ne le détecterait pas.
3. **Faible coût d'entraînement** : l'Isolation Forest s'entraîne en quelques secondes sur CPU, ce qui facilite les réentraînements périodiques.

IV.5.2 Configuration du modèle

Paramètre	Valeur	Signification
n_estimators	100	100 arbres d'isolation
contamination	0,05	5% d'anomalies attendues dans les données
max_samples	auto	256 échantillons par arbre (défaut sklearn)
max_features	1,0	Toutes les features utilisées par arbre
Features d'entrée	6 (NO _x , SO _x , PM _{2.5} , PM ₁₀ , CO ₂ , COV)	Polluants réglementaires uniquement

Pourquoi 6 features et non 8 ? La température et l'humidité sont exclues du vecteur IF car leur dynamique dépend de la météo extérieure — les inclure introduirait une source de variation qui n'est pas liée au processus industriel et diluerait la sensibilité aux anomalies de combustion.

IV.5.3 Règle de décision et sévérité

La fonction `decision_function` de sklearn retourne un score continu : les valeurs proches de 0 ou positives indiquent un profil normal ; les valeurs négatives indiquent une anomalie. La règle de décision est définie par un seuil configurable :

Règle de classification :

```
score < score_threshold (-0.20) → anomalie
score ≥ score_threshold          → normal
```

Sévérité graduée par distance au seuil :

```
score < -0.20                → Warning
score < -0.35 (seuil -0.15) → High
score < -0.50 (seuil -0.30) → Critical
```

Ce seuil est **le seul paramètre à ajuster** pour calibrer la sensibilité du détecteur sans réentraîner le modèle. Un abaissement à -0,22 augmente la sensibilité aux pics univariés (ex : NO_x ×5) au prix d'un léger risque de faux positifs.

IV.5.4 Problème du domain shift et solution retenue

Une décision de conception critique a été de ne pas utiliser les datasets publics d'air ambiant (EPA/UCI/Beijing) pour l'entraînement. Le tableau suivant illustre le problème :

Polluant	Plage EPA/UCI	Plage production tunisienne	Facteur
NO _x	0,02–0,08 ppm	374–680 mg/Nm ³	~10 000×
SO _x	0,001–0,03 ppm	187–340 mg/Nm ³	~5 000×
PM _{2.5}	5–35 µg/m ³	9,4–17 mg/m ³	~500×

Entraîner l’Isolation Forest sur les données EPA produisait **100% de faux positifs** : tous les profils de la cimenterie tunisienne étaient classés comme anomalies car le modèle n’avait jamais vu de telles concentrations.

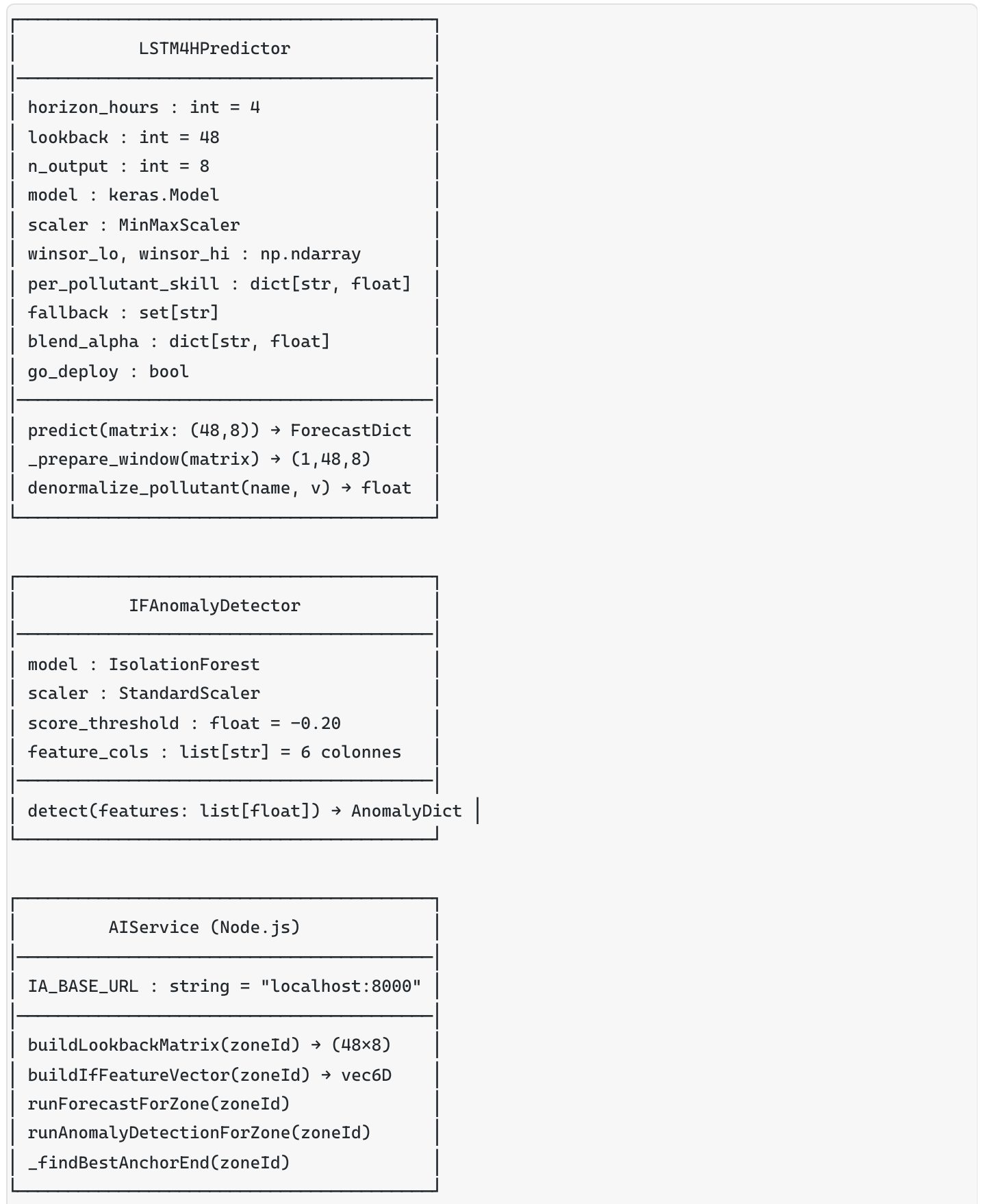
Solution : dataset hybride industriel synthétique. Un générateur (`generate_industrial_dataset.py`) produit des données synthétiques calibrées sur les niveaux réels du simulateur Node.js (`simulator.js`), en intégrant :

- 6 profils de zones avec multiplicateurs différents (Four, Broyage, Stockage, Expédition)
- Cycles opérationnels réalistes (double pic journalier 9h30 / 17h30, réduction weekend)
- Corrélations inter-polluants (CO₂ → NO_x, NO_x → SO_x, PM_{2.5} → PM₁₀)
- 5% d’anomalies injectées (spikes, dérives, incohérences capteur)

Résultat : 0% de faux positifs sur les profils normaux du simulateur, 5,5% de taux de détection d’anomalies (cohérent avec les 5% injectés).

IV.6 Diagrammes UML de conception

IV.6.1 Diagramme de classes — Module IA

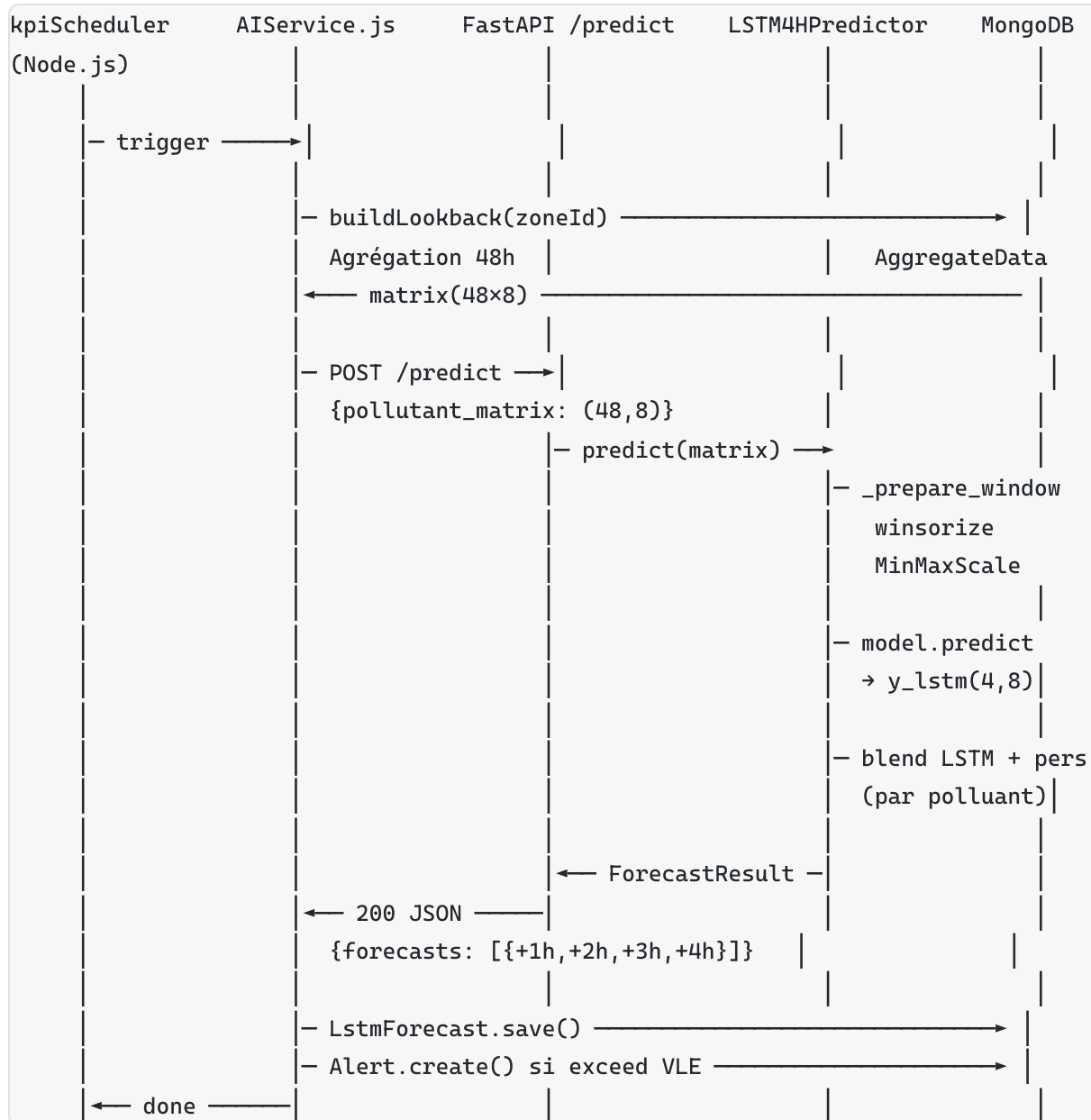


Collections MongoDB IA :

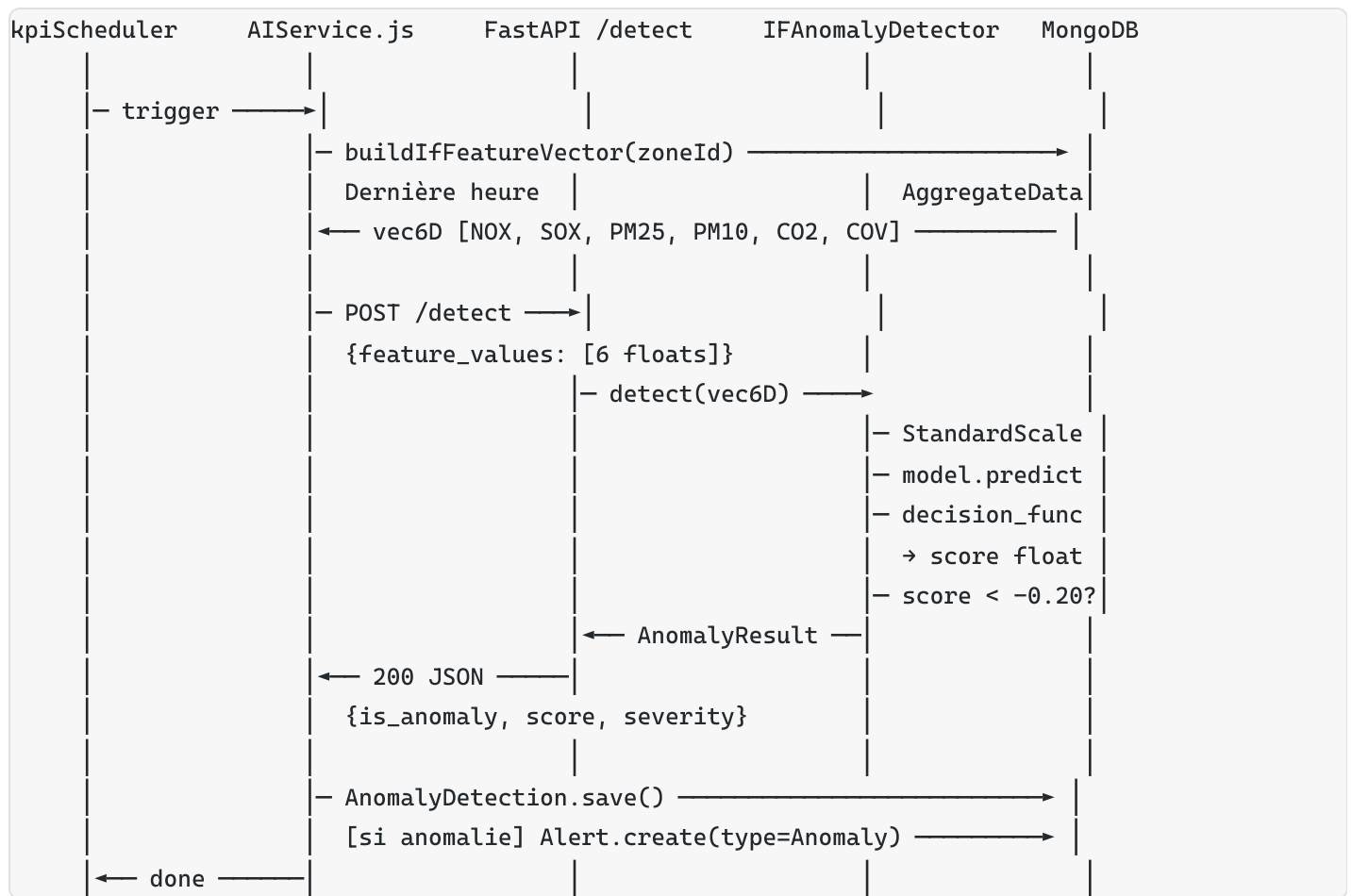
LstmForecast
zoneId
siteId
runAt
anchorPeriodStart
horizonHours
lookbackHours
steps[]
stepHours
pollutants[]
valuePhysical
severity

AnomalyDetection
zoneId
siteId
periodStart
isAnomaly : bool
anomalyScore : float
severity
featureValues[]
alertId

IV.6.2 Diagramme de séquence — Cycle d'inférence LSTM



IV.6.3 Diagramme de séquence — Détection d'anomalie IF



IV.7 Intégration backend — Architecture du service AIService

IV.7.1 Granularité zone et construction de la matrice LSTM

L'inférence est **zone-level** : chaque zone d'une industrie possède sa propre fenêtre de 48 heures, construite à partir des agrégats horaires de la collection `AggregateData` filtrés par `zoneId`. Cette granularité garantit que les profils spécifiques à chaque zone (Four de calcination vs Zone de stockage) ne sont pas mélangés.

`buildLookbackMatrix(zoneId) :`

1. Requête `AggregateData` :
`{ zoneId, period: "HOURLY",
periodStart: now - 48h, periodEnd: now }`
ordonnée par `periodStart` ASC
2. Pivot : une ligne = 1 heure, 8 colonnes = 8 polluants
`[CO2, NOx, SOx, PM2.5, PM10, COV, TEMP, HUMID]`
3. Si < 4 heures disponibles → `_findBestAnchorEnd()`
(recherche d'un ancrage historique sur 336 h)

Sortie : matrice numpy (48, 8) – valeurs physiques non normalisées
(la normalisation est effectuée dans `LSTM4HPredictor._prepare_window`)

IV.7.2 Endpoints IA du backend Node.js

Méthode	Endpoint	Description
GET	<code>/api/ia/health</code>	État des modèles (LSTM + IF chargés ?)
GET	<code>/api/ia/zone/:zoneId/forecasts/latest</code>	Dernier <code>LstmForecast</code> pour la zone
POST	<code>/api/ia/zone/:zoneId/forecasts/run</code>	Déclenche une inférence LSTM
GET	<code>/api/ia/zone/:zoneId/anomalies/history</code>	Historique <code>AnomalyDetection</code>
POST	<code>/api/ia/zone/:zoneId/anomalies/detect</code>	Déclenche une détection IF
POST	<code>/api/ia/retrain/dataset/prepare</code>	Prépare le dataset de réentraînement
POST	<code>/api/ia/retrain/start</code>	Lance le réentraînement
GET	<code>/api/ia/retrain/jobs/latest</code>	État du dernier job de réentraînement

IV.7.3 Alertes produites par le module IA

Quand l'IA détecte un dépassement futur prévu ou une anomalie actuelle, elle transmet les informations au backend Node.js qui crée une alerte dans MongoDB selon le même schéma que les alertes de seuil :

Source IA	Type alerte	Sévérité	Condition
LSTM — prévision	Forecast	Warning	Valeur prédite > warningThreshold
LSTM — prévision	Forecast	High	Valeur prédite > VLE
LSTM — prévision	Forecast	Critical	Valeur prédite > VLE × 1,5
Isolation Forest	Anomaly	Warning	score < -0,20
Isolation Forest	Anomaly	High	score < -0,35
Isolation Forest	Anomaly	Critical	score < -0,50

IV.8 Conception du pipeline d'entraînement et de réentraînement

IV.8.1 Pipeline d'entraînement initial (conception notebooks)

Phase 1 – Données

Notebook 01 : Collecte datasets publics EPA/UCI/Beijing
Notebook 02 : Nettoyage et normalisation
Notebook 03 / generate_industrial_dataset.py
: Génération dataset hybride industriel
: 6 sites × 90 jours × 1h = 103 680 lignes

Phase 2 – Entraînement

Notebook 04 : Isolation Forest

1. Pivot (site, heure) → colonnes NOX/SOX/PM25/PM10/CO2/COV
2. StandardScaler (fit sur train)
3. IsolationForest(n_estimators=100, contamination=0.05)
4. Évaluation : faux positifs sur profils normaux
5. Export : model_isolation_forest.pkl + if_scaler.pkl

Notebook 05 : Préparation tenseurs LSTM

1. Chargement CSV + pivot horaire par site
2. Fenêtres glissantes : $X(48, 8) \rightarrow y(4, 8)$
3. Split temporel 70/15/15 par site
4. Fusion des splits de tous les sites
5. Winsorisation (p1-p99 sur train)
6. MinMaxScaler [0,1] (fit sur train + target)
7. Export : lstm_train_val_test.pkl + lstm_scalers.pkl

Notebook 06 : Entraînement LSTM 4h

1. Chargement tenseurs depuis pickle
2. Construction modèle séquentiel (LSTM→LSTM→BN→Dense→Reshape)
3. Compilation : Huber pondérée, Adam(lr=0.001, clipnorm=1.0)
4. Callbacks :
 - SkillVsPersistence → val_skill à chaque epoch
 - EarlyStopping(monitor=val_skill, patience=15, mode=max)
 - ReduceLROnPlateau(factor=0.5, patience=4)
 - ModelCheckpoint (meilleur val_skill)
5. Entraînement arrêté à l'epoch 54 (meilleur = epoch 39)
6. Évaluation go/no-go → go_deploy = true
7. Export : model_lstm_4h.h5 + métriques + rapport skill

Phase 3 – Validation

analyze_simulator_vs_models.py :

- IF : profils normaux → NORMAL ✓
- IF : spikes → ANOMALIE ✓ (sauf univarié modéré)

- LSTM : dénormalisation cohérente ($\pm 22\%$ max)
- LSTM : valeurs physiques dans plages attendues

IV.8.2 Stratégie de réentraînement automatique

Le système détecte la dérive des modèles via la comparaison de RMSE entre les prédictions et les mesures réelles. Si la dégradation dépasse **20%** du RMSE d'entraînement, un réentraînement est déclenché automatiquement sur les 7 derniers jours de données MongoDB.

Déclenchement réentraînement :

```
[Monitoring continu]
Si RMSE_production > RMSE_train × 1.20
    |
    ▼
    IARetrainJob.create({ status: "PENDING" })
    |
    ▼
    POST /api/ia/retrain/dataset/prepare
    → IADatasetSnapshot : export MongoDB → CSV
    |
    ▼
    POST /api/ia/retrain/start
    → Python scripts/retrain_if.py + retrain_lstm.py
    → Nouveaux modèles exportés dans models/
    |
    ▼
    Critères go/no-go → si PASS
    → Modèles mis à jour (remplacement hot-swap)
    → IARetrainJob.status = "COMPLETED"
```

IV.9 Étude comparative des approches dataset

Le choix du dataset d'entraînement est la décision de conception la plus impactante du module IA. Les deux approches testées produisent des résultats radicalement différents.

Critère	Dataset public EPA/UCI/Beijing	Dataset hybride industriel tunisien
Taille	179 000 lignes, 25 sites	103 680 lignes, 6 sites
Domaine	Air ambiant urbain	Émissions industrielles cheminée
Cohérence production	Nulle	Totale (calibré sur simulator.js)
LSTM skill global	< -50%	+6,65%
LSTM skill NO _x	-205%	+6,1%
LSTM skill PM _{2.5}	-370%	+23,7%
IF faux positifs	100%	0%
IF détection	Inutilisable	5,5% (cohérent avec 5% injectés)
Utilisable en production	Non	Oui

La cause fondamentale de l'échec des datasets publics est le **domain shift** : les concentrations industrielles en cheminée sont 100 à 10 000 fois plus élevées que l'air ambiant urbain. Le scaler appris sur les données EPA ne peut pas représenter les niveaux industriels tunisiens, et l'Isolation Forest considère tous les profils industriels comme des anomalies.

IV.10 Tableau de synthèse — Conception complète du module IA

Paramètre	Valeur retenue	Justification
Langage	Python 3.10+	Écosystème ML standard (TensorFlow, sklearn)
Framework API	FastAPI (port 8000)	ASGI, Pydantic, OpenAPI intégré
Modèle prédiction	LSTM 2 couches (64+32)	Dépendances long terme, corrélations multivariées
Entrée LSTM	(48, 8) — 48h × 8 polluants	2 cycles jour/nuit, tous polluants
Sortie LSTM	(4, 8) — +1h à +4h	Planification opérationnelle (shift)
Modèle anomalie	Isolation Forest	Non supervisé, multivarié, faible coût
Features IF	6 (NO _x , SO _x , PM _{2.5} , PM ₁₀ , CO ₂ , COV)	Polluants réglementaires uniquement
Dataset	Hybride industriel synthétique (103k lignes)	Domain shift résolu vs datasets publics
Cadence données	Horaire (agrégation des lectures 30s)	Aligné KPI schedulers backend
Normalisation	MinMaxScaler LSTM + StandardScaler IF	Adapté à chaque algorithme
Loss	Huber pondérée par polluant	Robustesse outliers + priorité réglementaire
Fallback	Persistance si skill ≤ 0 par polluant	Garantie de ne pas dégrader vs naïf
Seuil IF	−0,20 (configurable sans réentraînement)	Calibrable selon sensibilité opérationnelle
Granularité	Zone-level	Profils spécifiques à chaque zone préservés
Réentraînement	Automatique si RMSE dégradé > 20%	Adaptation à la dérive des données production
Latence IF	~20 ms	Temps réel

Paramètre	Valeur retenue	Justification
Latence LSTM	~90 ms (après chargement modèle)	Acceptable pour planification shift